SIMATS ENGINEERING

Short Answer Test

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 30%

Sl. No	Questions	Bloom's Level
1	A startup team relies on random testing methods, leading to inconsistent results and missed defects. The project manager wants to improve testing effectiveness. As a test engineer, apply the concept of testing as an engineering activity to redesign the approach. Explain how structured test design improves quality and defect detection.	3
2	A banking system undergoes multiple failures due to lack of a defined testing process. The organization plans to standardize its testing activities. Illustrate how a well-defined testing process improves software quality in such real-time systems. Explain the benefits with relevant examples.	3
3	During development of an e-commerce platform, testing activities are performed inconsistently across phases. This leads to integration and system-level defects. Demonstrate how testing should be carried out as a structured process in SDLC. Explain how process discipline ensures better quality outcomes.	3
4	A login system shows mismatch between expected and actual outputs during validation. The development team is unsure whether the issue is an error, defect, or failure. Apply basic testing definitions to classify the issue. Explain the relationship between these terms with suitable examples.	3
5	A team struggles to detect defects early, leading to increased cost during later stages. The manager wants to improve defect detection efficiency. Apply a software testing principle to address this issue. Explain how the principle improves testing effectiveness with an example.	3
6	During requirement analysis of a healthcare system, several ambiguities go unnoticed. Later, these lead to defects in implementation. Show how a tester can contribute during the requirement phase to improve quality. Explain the importance of early testing involvement.	3
7	A payroll system produces incorrect outputs due to unclear and ambiguous requirements. The development team needs to identify the origin of defects. Analyze how such defects originate in the software lifecycle. Explain how requirement clarity can prevent such issues.	3

8	A system crashes when boundary input values are entered by users. The team suspects improper testing coverage. Classify this defect and justify your answer. Explain how boundary value analysis could have prevented this issue.	3
9	A company maintains a defect repository but rarely uses it for improving testing practices. This results in repeated defects across releases. Demonstrate how a defect repository can support better test case design. Explain its role in improving software quality.	3
10	An ATM system fails during withdrawal due to incorrect handling of transaction logic. The issue was not detected during testing. Apply your testing knowledge to identify and describe this as a functional defect. Explain how it could have been prevented.	3
11	A large application shows that most defects are concentrated in a few modules. The testing team wants to optimize their efforts. Use the concept of defect clustering to identify testing priorities. Explain how this improves testing efficiency.	3
12	A web application experiences slow performance under heavy user load. The issue needs to be categorized properly. Apply defect classification techniques to identify this type of defect. Explain how classification helps in better resolution.	3
13	A team is preparing for regression testing after multiple releases. They have access to historical defect data. Show how this data can be reused effectively for regression testing. Explain how it improves test coverage and efficiency.	3
14	During integration testing, multiple defects are found due to poor communication between developers and testers. The organization wants to improve collaboration. Identify the role of a tester in minimizing such defects. Explain how early involvement helps.	3
15	A software product repeatedly faces similar defects in every release cycle. The team needs a long-term solution. Apply defect prevention techniques to address recurring issues. Explain how prevention improves overall software quality.	3
16	A mobile application frequently crashes due to inconsistent testing approaches. The testing team wants to standardize practices. Apply testing principles to redesign the testing strategy. Explain how structured approaches improve reliability.	3
17	A company confuses debugging with testing, leading to delays in defect resolution. The team needs clarity on both concepts. Differentiate testing and debugging using a real-time scenario. Explain their roles in improving software quality.	3
18	An online shopping system behaves differently across browsers. This leads to customer dissatisfaction. Apply testing principles to design a cross-browser testing strategy. Explain how such issues can be effectively handled.	3

19	A logistics application produces inconsistent outputs due to improper validation. The issue is not detected early. Apply testing concepts to identify gaps in validation. Explain how proper testing improves system reliability.	3
20	A team skips structured test case design and directly executes random tests. This leads to poor defect detection. Apply testing as an engineering activity to improve the process. Explain the importance of systematic test design.	3
21	A healthcare system shows data inconsistency between modules due to integration issues. The testing team needs to analyze the problem. Apply testing principles to identify root causes. Explain how proper testing ensures data consistency.	3
22	A project experiences delays because testers are not involved early in development. The organization wants to improve collaboration. Explain the role of testers across SDLC phases. Show how early involvement improves quality.	3
23	A system fails during peak usage due to untested scenarios. The testing team missed critical cases. Apply testing principles to identify gaps. Explain how better test coverage prevents such failures.	3
24	A software team lacks proper defect documentation, making tracking difficult. The organization wants to improve defect management. Apply defect repository concepts to solve this issue. Explain how documentation enhances tracking and analysis.	3
25	A banking system produces repeated transaction failures due to improper testing practices. The team needs to improve quality. Apply software testing principles to redesign the testing approach. Explain how structured testing enhances reliability.	3
26	A system shows failures due to missing edge case testing. The testers focused only on normal inputs. Apply testing principles to include edge cases. Explain how boundary testing improves defect detection.	3
27	A development team struggles to maintain consistency in testing across modules. This leads to uneven quality. Apply process-based testing approaches to address this issue. Explain how standardization improves outcomes.	3
28	A web application fails due to improper handling of unexpected inputs. The testing team missed validation scenarios. Apply testing concepts to improve input validation. Explain how proper testing ensures robustness.	3
29	A software company faces frequent defect leakage into production. The management wants to reduce such issues. Apply testing principles to analyze the root cause. Explain how improved testing practices prevent leakage.	3
30	A project team is unable to identify defects early due to poor testing strategies. This increases overall cost and effort. Apply testing principles to redesign the strategy. Explain how early defect detection improves efficiency.	3

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**25**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**20**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**10**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

CO1 - AT2

NAME: PAVITHRA.C REG NO: 192411027

1. Testing as an Engineering Activity

Software testing should follow a systematic and planned approach instead of random testing

Requirement analysis: understand the software requirements

Test planning: Define objectives, resources, schedule and testing methods.

Test case design: prepare test cases with inputs, excepted outputs and conditions

Test execution: Execute test cases and compare actual results with expected results

Example: In an e-commerce application, test - login, product search, cart and payment using planned test cases

2. Importance of well - Defined Testing process

A well - defined testing process ensures that software is tested systematically and consistently

⇒ identify defects at an early stage

⇒ Improves reliability and security

⇒ Ensures all important requirement are tested

⇒ Reduces development and maintenance costs

⇒ provides proper documentation and tracking

Eg: banking system

RESULT: A standardized testing process reduces failu
and improves customer confidence

3. Testing as a structured process in SDLC

Testing should be performed throughout the software Development life cycle (SDLC)

Requirement analysis: Identify testable requirements

Test planning: Decide testing methods, resources and schedule.

Test design: prepare test cases and test data

Test environment setup: configure the required hardwa
and Software.

Example: E-commerce platforms

RESULT: Following the SDLC testing process improves defect detection, integration quality and overall software reliability.

9. Role of a Defect Repository

A defect repository is a centralized storage system used to record and manage software defects

⇒ stores defect descriptions: severity, priority and status

⇒ Maintains historical defect information

⇒ Helps to identify frequently occurring defects

⇒ Supports the design of effective test cases.

RESULT: A defect repository improves test coverage defect tracking and software quality.

10. Functional Defect in an ATM System

=> A functional defect occurs when software does not perform a required function correctly.

Eg: When a customer withdraws ₹2,000 the ATM deducts ₹3,000 from the account or dispenses cash

prevention:

Test valid and invalid withdrawal amounts
verify account balance updates
Test insufficient balance scenarios

RESULT: Funtional testing helps detects transaction errors and ensures accurate, reliable ATM operations

11. Defect clustering

Defect clustering is the principle that most defects are often concentrated in a small number software modules.

=> Analyze previous defect reports
=> identify modules with frequent defects
=> prioritize high-risk modules for testing
=> Allocate more testing resources to those modules.

RESULT: Defect clustering improves testing efficiency prioritization and defect detection

18. Cross - Browser Testing strategy

Cross browser testing verifies that a web application works consistantly across different browsers and devices

⇒ Identify supported browsers, such as chrome Firefox, Edge and Safari

⇒ Prepare test cases for important functions

⇒ Test page layouts, buttons, forms, navigation

RESULT: Cross - browser testing improves compatibility, usability and customer satisfication

22. Role of Testers Across SDLC Phases

Testers should participate in every stage of the SDLC rather than Jaining only after deployment

Example: In a banking application, a tester identifies an unclear withdrawal limit during requirement analysis, preventing incorrect implementation

RESULT: Early tester involvement reduces rework improves collaboration.

19. Importance of Test coverage During peak usage

Test coverage measures how much of the software requirements code or functionality has been tested

Eg: online shopping

RESULT: Better test coverage and performance testing reduce failures and improve System reliability during peak usage.

20 Boundary Testing and Edge

Boundary value Analysis (BVA) is a testing technique that checks values at the boundaries of an input range because defects frequently occur at these limits

Input	Expected RESULT
17	Invalid
18	valid
19	valid